ACM Turing Award Nomination

Beresford N. Parlett

8 May 1987

This letter supports the nomination of Prof. W. Kahan for the Turing award. The case rests on his contribution over three decades to the understanding of COMPUTER ARITHMETIC.

1 Overview

Kahan exposes the perils of using poorly designed arithmetic engines (see Section 3) and elucidates the unremarked benefits, at the programming level, of clean arithmetic (see Section 2). An outward and visible sign of the service he has rendered to those who have crunched numbers, or will crunch them in years to come, is the IEEE Standard for Floating Point Binary Arithmetic, P745, and its radix free version, P854. Although it embodies the work of many people, no one who took part in the gruelling deliberations would deny that Kahan was the architect of these standards.

These documents reach farther and innovate more than was envisaged by those chip designers who innocently requested an arithmetic standard more than ten years ago. It is not obvious that P754 will benefit scientific computation nor that any committee's output could be an intellectual achievement of the first magnitude. We speak to these doubts in the next section whose goal is to indicate, in the briefest way, the truth of the claims without burying the reader in unwelcome detail.

Here we suggest why Kahan dominates the study of computer arithmetic, why he is not replaceable, and why he is unlikely to have a successor. His mastery can be attributed to three qualities:

1. Confidence in his ability as a mathematical *analyst* of top quality. As an undergraduate, he was part of the University of Toronto's winning (among top five) Putnam team. His mathematical output is small but striking.

2. Deep knowledge of arithmetic *hardware* as well as the pertinent aspects of operating systems and compilers. He knows how much each feature will cost both in performance and in design time.

3. Knowledge of a staggering variety of *applications*, i.e. specific computational problems, and their sources.

A number of people can match or exceed Kahan in either of the first two qualities. I have yet to hear of anyone with pretensions to rival him in both of them, let alone all three. He was fortunate to start his career in the 1950's when computer science itself was in its infancy. Few young people today have the time for training in engineering, mathematics, and computer science, nor the opportunity to work in a university computer center assisting everyone encountering technical difficulties. "We shall not see his like again."

Kahan has made two other significant contributions to Computer Science, both unconventional and neither on its own warranting a major award. The first is his persistent and sometimes effective exposure of bad design in arithmetic units. The second is the quality of the algorithms behind three remarkable keys on some hand held calculators: INTEGRATE, SOLVE, and IRR (Internal Rate of Return). More information is given in the section following the IEEE standards.

Kahan works in the sewers of Computer Science (his own description) trying to keep them free from the efluents of those who think they know how arithmetic should be done but have not pondered all the consequences. It seems likely that the rest of his career will be consumed with the attempt to persuade the makers of operating systems and programming languages to allow users to reap the rewards — in simplicity and performance — latent in the IEEE floating point arithmetic standards.

The force of his mind impresses all who come into contact with him. Far more of his formidable energy has gone into keeping up the quality of computing devices than into enhancement of his own career.

2 The IEEE Standards for Floating Point Arithmetic

The VLSI phenomenon held out the possibility of putting onto one or two chips an arithmetic engine (FPU) which could be sold, eventually, for under one hundred dollars. Here was an opportunity to start afresh incorporating

into the specifications most of what had been learnt during the first twenty-five years of the computer era.

The IEEE Committee began its work in 1977 with no intimation that its deliberations would rumble on for five strenuous years. In fact, the standard for binary floating point arithmetic, P754, was not official until 1985. The radix free version, P854, was finally approved in 1987.

A casual glance at P754 itself could raise some questions.

(A). Technical standards frequently represent compromises between contending commercial parties. The product outcome may perhaps be useful, but is unlikely to be intellectually distinguished. What is special about arithmetic?

(B). P754 and P854 are unexpectedly complicated and incorporate notions which, though not actually untried, are new to most readers. Perhaps these standards are academic excesses to be deplored rather than a quite extraordinary achievement to be honored?

A brief answer to (A) is that the experts, the designers of arithmetic units, had a great deal to learn about the implications of their decisions for users of their products. This area is still not well understood. A brief response to (B) is that the apparent innovations survived three or four years of intense debate. There was no spirit of "that's a neat idea, let's try it." The best revolutionaries are reluctant revolutionaries.

These assertions need some justification.

None of the features in the standard was "invented" by Kahan, yet all of them have been touched by him in the process of thinking through all the implications of mutual interaction.

The standard appears complex but might more aptly be described as detailed. It aims to be "tight," to specify outcomes whenever uniqueness can be seen to be warranted in all situations. Coherence at this layer will permit portability of programs at higher levels. Indeed it is not outrageous to say that this tightness is *necessary* if sharing of code (as against sharing of methods or ideas) is to become a habit in the world of scientific computing.

In a machine that conforms to the standard, the obvious procedure for calculating the Euclidean norm of a vector is the right one. Contrast this with the complicated program published by J. Blue (See James L. Blue's "A Portable Fortran Program Defined the Euclidean Norm of a Vector," *ACM TOMS*, Vol. IV, No. 1, March 1978, p. 15.) and designed to run on all of today's machines without suffering from underflow or overflow. Moving

from current FPUs to conforming FPUs is a little like moving from Roman numerals to the positional (Arabic) system.

We consider some features of the specifications in P754.

Correct Rounding

There was no controversy about this requirement. Yet it is worth noting that none of the older designs embodied in IBM 360, CDC 7600, nor the Cray and Cyber machines rounds correctly or even consistently. Apparently, the extra cost did not seem warranted to their designers, so the easy agreement on this issue is not without interest.

The standard also requires directed rounding. Among other things, this permits efficient implementation of interval arithmetic. Thus, P754 is concerned with the future as well as the present.

Gradual Underflow (instead of flush-to-zero)

In retrospect, the controversy over this requirement was grossly overblown. Now it seems natural to go with the flow. The performance cost is significant for the top-of-the-pipeline products that seek to approach CRAY performance, but is not an issue for the rest.

Consider division of two representable complex numbers. This is not a primitive operation of the system and so the result does not have to be correctly rounded.

CLAIM: if no exception occurs in executing Smith's formula (see D. Kunth's *Art of Programming*, Vol. II, p.195) except for underflow (or inexact result), then the relative error in the result is bounded by 10×2^{-24}, given gradual underflow, but only by 2^{-1}, given flush to zero. The roundoff unit is 2^{-24} for single precision.

(See note by D. Hough in the minutes of the committee for April 17, 1980). These bounds are quite realistic.

What is clever here? Answer: Producing the illuminating example, namely complex division.

It is sad, but not surprising, to relate that Donald Knuth's handsome letter of April 28, 1980 had more effect on the committee than did the technical arguments. Gradual underflow disagrees with the view expressed in the first edition of Knuth's book. He was nice enough to call gradual underflow a brilliant invention. It makes underflow look like roundoff error — a big step towards conceptual simplicity.

The previous remarks disguise the complexity of the situation. Why not introduce a special symbol for underflow? Kahan himself had proposed this in 1966 and Fraley and Walter (of Hewlett-Packard) independently advanced this proposal in 1978. Perhaps it is just as good as gradual underflow? Once again the challenge was to find an example that will discriminate. Kahan produced a function f, involving an integral, such that both gradual underflow and flush-to-zero produce the proper output (+41.59) whereas use of a symbol yields -2^{-67}. Here is the function

$$f(x,y,z,t) = \begin{cases} \frac{x}{yz} - \frac{\sin(1/z)}{t+\int_0^{z^2}(\sin u/u)du}, x\sqrt{t} \geq y \\ \ln[1 + \frac{(z-1)}{1+\int_{z^2}^{y^2/t}(\sin u/u)du}], \text{otherwise} \end{cases}$$

Here are specific arguments: $x = U$, $y = 2^{-64}$, $z = 4/y$, $t = 2$. Here U is the symbol and the underflow threshold is 2^{-126}.

How did he come up with that example?

INFINITY

It took so long for mathematicians to welcome the symbol ∞ that we should not berate computer manufacturers for their caution. CDC introduced an infinity symbol in 1964. It is unfortunate the FORTRAN decrees that division by zero is a capital offence. This puts a damper on the use of continued fractions which is a powerful tool in approximation of functions.

The standard requires the presence of signed ∞ (affine mode) but Kahan lost a final vote on whether projective mode ($1/0 = -1/0$) should be provided as well. Of course, signed ∞ brings with it signed zero.

NaN (Not-a-Number)

This feature had been used before (CDC), but the standard takes it up, uses it consistently and to far more effect than before. The idea is striking. The existence of NaNs allows a calculation (in particular an inner loop) to proceed without branching despite the appearance of exceptions such as $0/0$, $(+\infty)-(+\infty)$, or uninitialized variable. The value of a NaN can be a pointer to the first occurrence of an anomaly. Thus, retrospective diagnostics are possible.

NaNs are wonderful in many contexts, but they do spoil the total ordering of the number system and imply that floating point comparison works

slightly differently from integer comparison, to the annoyance of some compiler writers.

3 Watch Dog Activities

Kahan has been a formidable but fairminded critic of arithmetic units that treat the last few bits in their numbers in a sloppy way. He has been more aware than most people of the subtle and surprising consequences of what he calls dirty arithmetic. In particular, he is aware of the penalty paid in writing and debugging software that *may* be implemented on such facilities. The high quality of the arithmetic operations in the conventional IBM and DEC mainframe computers is, in no small way, due to Kahan's persistent exposure of delinquents.

Very few people are capable of this detective work. It does not fit into the conventional framework of research in computer science and he has had no public recognition for his labors.

Below we give three examples.

A.

Kahan's persistent elaboration of the consequences of omitting a guard digit in IBM's 360 machines helped to persuade that company to make expensive modifications to its machines including those that had been already installed.

B.

In approximately 1979, Texas Instruments took a full page advertisement in the New York Times to extol the virtues of their new business calculator. They chose to illustrate its prowess on a (complicated) bond sale. Who but Kahan would have thought to verify the numbers used in the advertisement? Sure enough they were wrong. The discrepancy was small, about \$30 in $\$10^8$, but that is enough to raise questions as to whether calculators can be used in legal evidence. The discrepancy was unnecessary and caused by a logarithm procedure that allowed several units error in the last place held for arguments close to unity.

Other surprising and subtle consequences of employing less than impeccable arithmetic units are described in the unpublished report "Can You Count on Your Calculator?" There is an example where execution in TI's

13 decimal digit arithmetic gives worse results than in clean 10 decimal digit arithmetic. That is deeply counter intuitive.

C. ACRITH

This system was put on the market by IBM in 1984. It purports to relieve the naive user of the need to worry about error analysis, ill conditioning, or numerical analysis.

The key idea is that all numerical computations can, in principle, be cast into the form of a sequence of dot products of vectors. If a conventional arithmetic system is embellishment with an exact (arbitrary length) dot product operation then, in principle, all our numerical worries are over.

ACRITH is a nice example of "gee, that's a nice idea, let's implement it." Quite the opposite of the IEEE standard which underwent five years of criticism and debate. While the dot product formulation is quite suitable for some computations, it is singularly inappropriate for others. No consideration is given to the time and storage penalty for this reformulation. The failings of this system are veiled both by industrial secrecy and a heavy handed volume of theory which provides the "justification" for the system.

Kahan and LeBlanc (a student at UCB) published a beautiful, short paper describing the anomalies they managed to discover — without benefit of any specifications of the arithmetic system. The package fails in some innocuous problems that cause no problems to conventional arithmetic units. However, it can also compute correct results on horrendously ill-conditioned problems. For example, using only 16-decimal numbers, it inverts a 21×21 Hilbert matrix whose condition number is 10^{29}. Is this a fluke? Unfortunately, yes. Kahan, together with his student Jim Demmel, tried permuting some columns of the matrix and the package refused — as is reasonable. ACRITH also refused to invert a well-conditioned 7×7 matrix. The brilliance lies in finding simple examples that defeat this so-called foolproof system.

4 ALGORITHMS

In lieu of payment for consulting work on the early HP scientific calculators, Kahan asked that two keys be added to the next generation calculator.

INTEGRATE approximates $\int_a^b f(s)ds$ using techniques and tricks that have not yet found their way into any numerical analysis textbooks. Of

course, they compute an error estimate as well as the integral. The program uses 6 cells of work space in addition to the inputs.

The SOLVE key takes two guesses, a, b and f, and returns a tiny interval that either contains a sign change in f or a local minimum. There is no need for $[a, b]$ to contain a zero. Indeed, f need not be continuous nor have a zero. Again the working space is minute.

Although it is not appropriate to ask recognition for algorithms that are not yet in the open literature, it is not unreasonable to point to their performance on examples that bring many a library subroutine to its knees. After some careful investigations, a V/P at Bechtel required all his engineers to use HP calculators.

The IRR key (Internal Rate of Return on a sequence of transactions) is required to find the unique positive zero of a high degree polynomial. The initial specifications, from HP, wanted the code to work on cases with degrees up to 2000. (Kahan has said that a polynomial of degree 2000 has zeros almost everywhere.) For those who work on zero finding, that seems an impossible challenge. In fact, a later version of this algorithm dealt with a polynomial of degree over a million! I have never heard of a comparable instance in the computation of polynomial zeros.